DSP in VLSI

Final Project

Eigen-value Decomposition by Iterative QR Operations

Student ID

M11407439 / B11107027

Name

MING-HONG, LIN / YING-CHENG, CHEN

Development Environment

HDL	SystemVerilog
Simulation Tools	Synopsys VCS and Verdi
Synthesis Tool	Synopsys DC Compiler
Process	TSMC 16nm (ADFP)
Verification	Python

Due Date: 2026/06/17 05:00 PM

Contents

1 Section 1: Wordlength and RMSE Analysis **2**
- 1.1 Minimum Fractional Wordlength & CORDIC Stage Selection . . . 2
- 1.2 Integer Bit-Width Analysis . . . 3
- 1.3 Fixed-Point Bit-True Model . . . 4
- 1.4 CORDIC Scaling Factor . . . 4

2 Section 2: Bit-True Model Verification **5**
- 2.1 RMSE Per Pattern . . . 5

3 Section 3: Hardware Architecture and Control **6**
- 3.1 CORDIC PE Architecture . . . 6
- 3.2 Systolic Array-Based QRD Architecture . . . 7
- 3.3 EVD Architecture . . . 8

4 Section 4: Hardware Simulation **10**

5 Section 5: Fixed-Point vs. Hardware Consistency **11**

6 Section 6: Area-Time Product **12**

7 Section 7: Design Innovations and Team Contributions **13**
- 7.1 Team Contributions . . . 13

Section 1: Wordlength and RMSE Analysis

1.1 Minimum Fractional Wordlength & CORDIC Stage Selection

To set the fixed-point parameters of the EVD bit-true model, we sweep the fractional wordlength and the number of CORDIC micro-rotation stages against all 11 matrices in `Final11Pattern.mat`. A configuration is accepted only when the RMSE of eigenvalues and eigenvectors both remain below 10^{-2} for every test pattern:

$$\mathrm{RMSE}_\lambda = \sqrt{\frac{1}{3}\sum_{i=1}^{3}\left(\hat{\lambda}_i - \lambda_i\right)^2} \quad \mathrm{RMSE}_v = \sqrt{\frac{1}{9}\sum_{i=1}^{3}\left\|\hat{\mathbf{v}}_i - \mathbf{v}_i\right\|_2^2}$$

where λ_i and $\hat{\lambda}_i$ are the reference and bit-true eigenvalues, and $\mathbf{v}_i$ and $\hat{\mathbf{v}}_i$ are the associated eigenvectors.
We use a two-step search rather than tuning both knobs at once. First, with the CORDIC stage count fixed at 8, we increase the fractional wordlength b until the RMSE criterion is met on all 11 sets; the smallest such value is $b = 11$. Second, with $b = 11$ held fixed, we sweep the CORDIC stage count and find that 8 stages is again the minimum setting that satisfies the same criterion. Thus, the selected bit-true configuration is **11 fractional bits** and **8 CORDIC stages**, which are adopted consistently in the subsequent verification and hardware implementation.

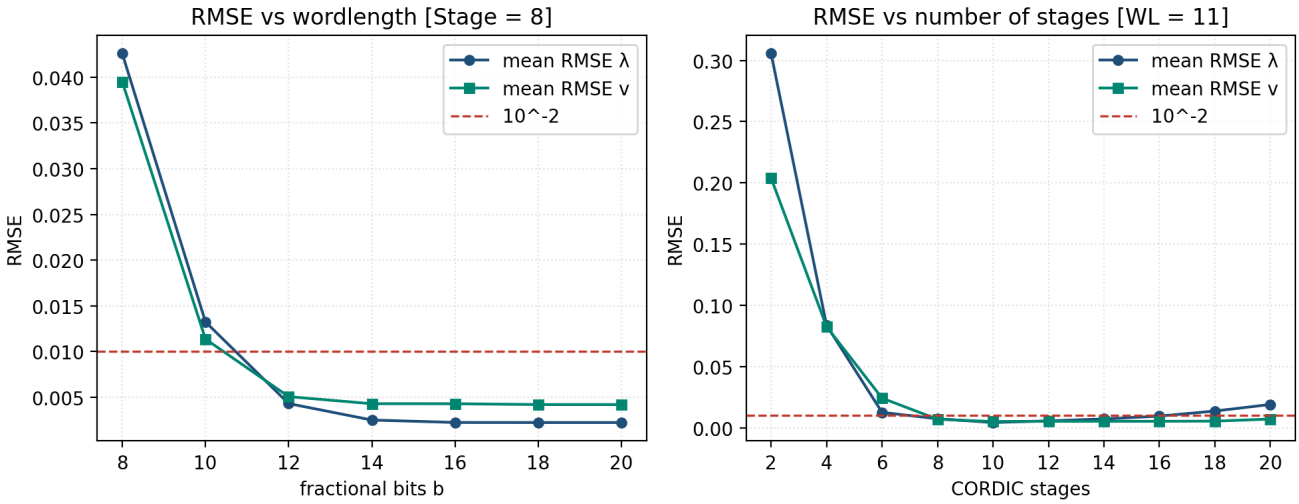

Figure 1.1: RMSE vs. fractional bits ($N_{\text{stage}} = 8$, left) and CORDIC stages ($b = 11$, right).

The two-step sweeps above are useful for intuition, but they do not jointly identify the smallest (b, N_{stage}) pair. Each pass fixes one parameter at the value found earlier, so a configuration that fails in isolation may still satisfy the RMSE criterion when both knobs are set together, and the true minimum combination may be missed.

We therefore perform a two-dimensional sweep over fractional bits b and CORDIC stages N_{stage}. Every grid point is evaluated on all 11 test patterns; a setting passes only when both RMSE_λ and RMSE_v remain below 10^{-2} for every set.

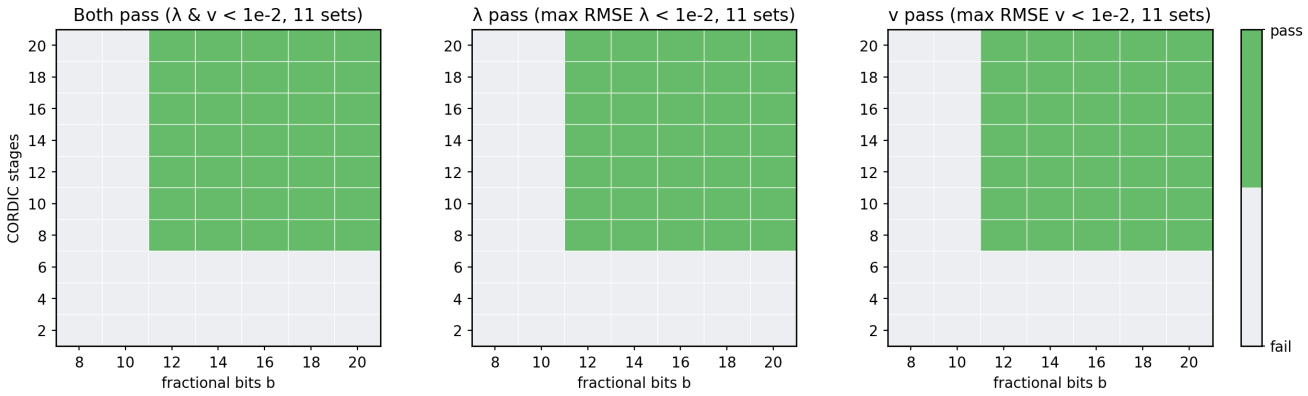

Figure 1.2: 2D pass maps over b and N_{stage} (11 test patterns).

Table 1.1 summarizes the hardware configuration selected from the sweeps above.

Table 1.1: Hardware Configuration

Parameter	**Value**
`B` (fractional bits)	11
`N_STAGES` (CORDIC stages)	8
`MAX_ITER` (max iterations)	7

1.2 Integer Bit-Width Analysis

With the configuration in Table 1.1, we record the dynamic range of each EVD stage across all 11 patterns and derive the minimum integer wordlength needed at each node. Table 1.2 lists the worst-case integer bit requirement and the corresponding fixed-point format (1 sign bit + integer + B fractional bits).

Table 1.2: Integer Bit-Width Analysis

Node	**Min**	**Max**	**Max abs**	**Int. bits**	**Q format**	**Total**
Quantized input	−2.394	10.239	10.239	4	Q4.11	16
Matrix T/R	−5.117	10.585	10.585	4	Q4.11	16
Eigenvector U	−0.981	1.000	1.000	1	Q1.11	13
CORDIC input	−5.117	10.585	10.585	4	Q4.11	16
CORDIC internal	−10.586	17.432	17.432	5	Q5.11	17
Scaled output	−5.117	10.585	10.585	4	Q4.11	16

1.3 Fixed-Point Bit-True Model

With the fractional wordlength, CORDIC stage count, iteration limit, and integer bit-width requirements established above, we implement a fixed-point bit-true model of the iterative QR EVD datapath. The model applies the formats in Tables 1.1 and 1.2 at each quantization point so that its arithmetic matches the behavior targeted by the RTL design.
This model supports more accurate fixed-point error analysis than a floating-point reference, because quantization, CORDIC rounding, and iterative accumulation are modeled explicitly. It also supplies golden eigenvalues and eigenvectors for RTL verification, so behavior and gate-level simulations can be compared against a consistent fixed-point reference instead of double-precision software alone.

1.4 CORDIC Scaling Factor

Sections 1.1–1.2 do not analyze the impact of quantizing the CORDIC scaling factor. They model gain compensation as $Q_B(X/K)$ only (divide by ideal K), then quantize the quotient. This is intentional, we do not want scaling-factor precision to enlarge global B and widen the entire systolic datapath. Therefore, Extra bits are spent only at this multiply-and-round point, not across the systolic array.

Table 1.3: CORDIC Scaling Factor Configuration

Parameter	**Value**
`DATA_FRAC` (datapath fractional bits)	11
`DATA_WIDTH` (datapath total bits)	17
`K_INV_FRAC` (fractional bits for $1/K$)	16
`K_INV_DW` (total bits for $1/K$)	17

Section 2: Bit-True Model Verification

2.1 RMSE Per Pattern

We evaluate the bit-true model on all 11 patterns in `Final11Pattern.mat` with $B = 11$, $N_{\text{stage}} = 8$, and MAX_ITER = 7. Figure 2.1 plots RMSE_λ and RMSE_v per pattern; both remain below 10^{-2}.

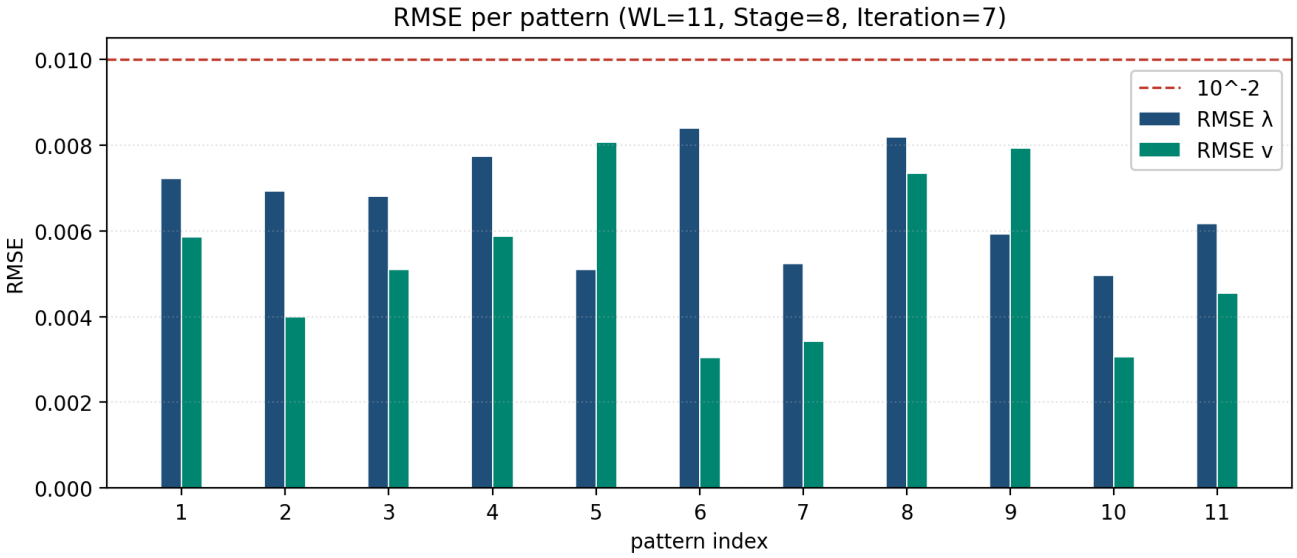

Figure 2.1: RMSE per pattern.

Section 3: Hardware Architecture and Control

3.1 CORDIC PE Architecture

We follow the Homework 4 CORDIC PE design: inputs are preconditioned in the initial stage, and the eight CORDIC micro-rotation stages are unfolded and pipelined to maximize throughput. For this EVD application, **the exact rotation angle θ does not need to be stored.** Vectoring mode records each micro-rotation direction in a rotational direction register (RDR) and rotation mode replays those directions on the remaining matrix entries.
All θ-related arithmetic in the CORDIC datapath can therefore be removed, improving PPA (power, performance, and area). Figure 3.1 shows the resulting PE architecture.

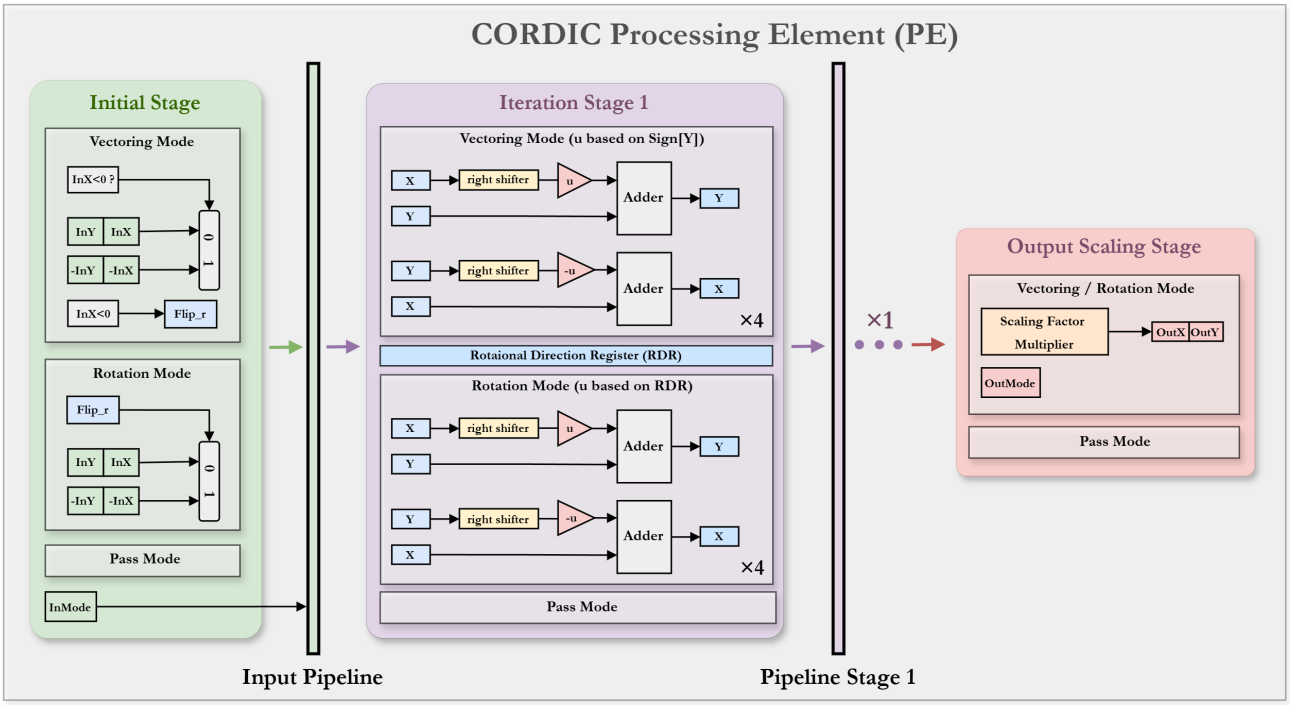

Figure 3.1: CORDIC PE Architecture.

3.2 Systolic Array-Based QRD Architecture

Each `CORDIC_PE` produces one result every two cycles, so the triangular QRD systolic array is retimed around this two-cycle latency. We remove the delay unit at the bottom-right diagonal position and route the vertical datapath directly into the output row, so horizontal and vertical streams stay aligned at the array outputs. The detailed timing schedule is given in the next section.
Figure 3.2 shows the resulting array: three `CORDIC_PE` blocks (2-cycle), one single-cycle `Delay_Unit`, and one two-cycle `Delay2_Unit`.

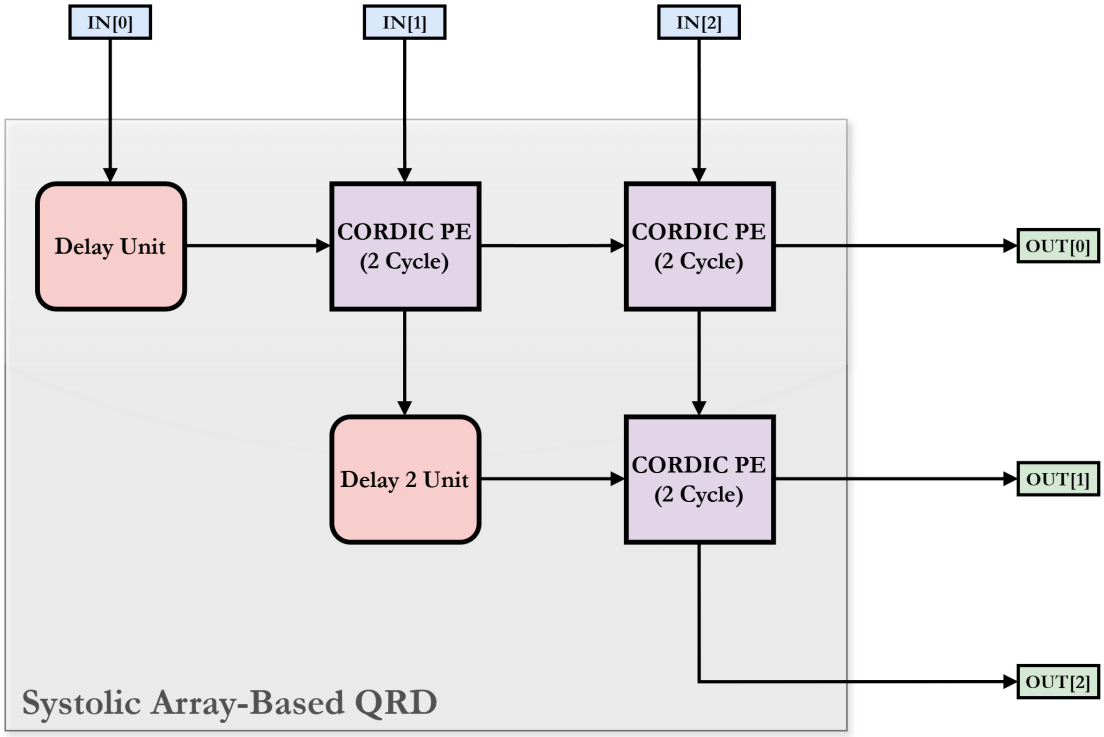

Figure 3.2: Systolic array-based QRD architecture.

3.3 EVD Architecture

The top-level `EVD` module wraps the QRD systolic array with an input delay unit (`IDU`), matrix registers, and a three-state FSM. Table 3.1 summarizes the control flow; Figure 3.3 gives the cycle-by-cycle hardware schedule.

Table 3.1: EVD FSM States

State	Function
`IDLE`	When `InValid` is asserted, stream the external 3×3 input matrix into the systolic array over three cycles, then enter `PROCESS`.
`PROCESS`	Run seven QR iterations. Each iteration takes 15 cycles: feed U for eigenvector update, feed R^{T} for T update, then feed the updated T back into the array.
`OUT`	After the seventh iteration, stream the eigenvalues and the three rows of U (eigenvectors) to the output ports over four cycles, then return to `IDLE`.

In `PROCESS`, the FSM counter (`cnt`) schedules matrix reads, `TEMP` buffering, and `U_REG` updates so that systolic inputs and outputs remain aligned with the two-cycle `CORDIC_PE` latency. Figure 3.3 tabulates this schedule for the `IDLE` load phase and one `PROCESS` iteration.

	Input Data			Output Data			Output Buffer					U Register								
Cycle	InD0	InD1	InD2	OutD0	OutD1	OutD2	0	1	2	3	4	[0][0]	[0][1]	[0][2]	[1][0]	[1][1]	[1][2]	[2][0]	[2][1]	[2][2]
0	T00	T10	T20									I	0	0	0	I	0	0	0	I
1	T01	T11	T21									I	0	0	0	I	0	0	0	I
2	T02	T12	T22									I	0	0	0	I	0	0	0	I
0	U00	U10	U20									I	0	0	0	I	0	0	0	I
1	U01	U11	U21									I	0	0	0	I	0	0	0	I
2	U02	U12	U22	R00								I	0	0	0	I	0	0	0	I
3				R01			R00					I	0	0	0	I	0	0	0	I
4				R02	R10	R20	R00	R01				I	0	0	0	I	0	0	0	I
5	R00	R01	R02	U00	R11	R21	R00	R01	R02	R10	R20	U00	0	0	0	I	0	0	0	I
6	R10	R11	R12	U01	R12	R22	R11	R21	R02	R10	R20	U00	U01	0	0	I	0	0	0	I
7	R20	R21	R22	U02	U10	U20		R21	R22	R10	R20	U00	U01	U02	U10	I	0	U20	0	I
8					U11	U21						U00	U01	U02	U10	U11	0	U20	U21	I
9					U12	U22						U00	U01	U02	U10	U11	U12	U20	U21	U22
10				T00								U00	U01	U02	U10	U11	U12	U20	U21	U22
11				T01			T00					U00	U01	U02	U10	U11	U12	U20	U21	U22
12	T00	T10	T20	T02	T10	T20	T00	T01				U00	U01	U02	U10	U11	U12	U20	U21	U22
13	T01	T11	T21		T11	T21	T00	T01	T02			U00	U01	U02	U10	U11	U12	U20	U21	U22
14	T02	T12	T22		T12	T22	T00	T01	T02			U00	U01	U02	U10	U11	U12	U20	U21	U22

Figure 3.3: Hardware timing schedule (`IDLE` input load and one `PROCESS` iteration).

Guided by the timing schedule in Figure 3.3, the three-state FSM acts as the `EVD` controller. It steers `InData` through an input buffer (data pre-processor and skew-aligned `IDU`), the systolic-array-based QRD core, and an output buffer (`TEMP` staging and `EV_REG`). During `PROCESS`, feedback from the output buffer closes the U and T update loop; in `OUT`, the controller streams eigenvalues and eigenvectors from `EV_REG` and `U_REG` to `OutData`. Figure 3.4 shows the resulting top-level architecture.

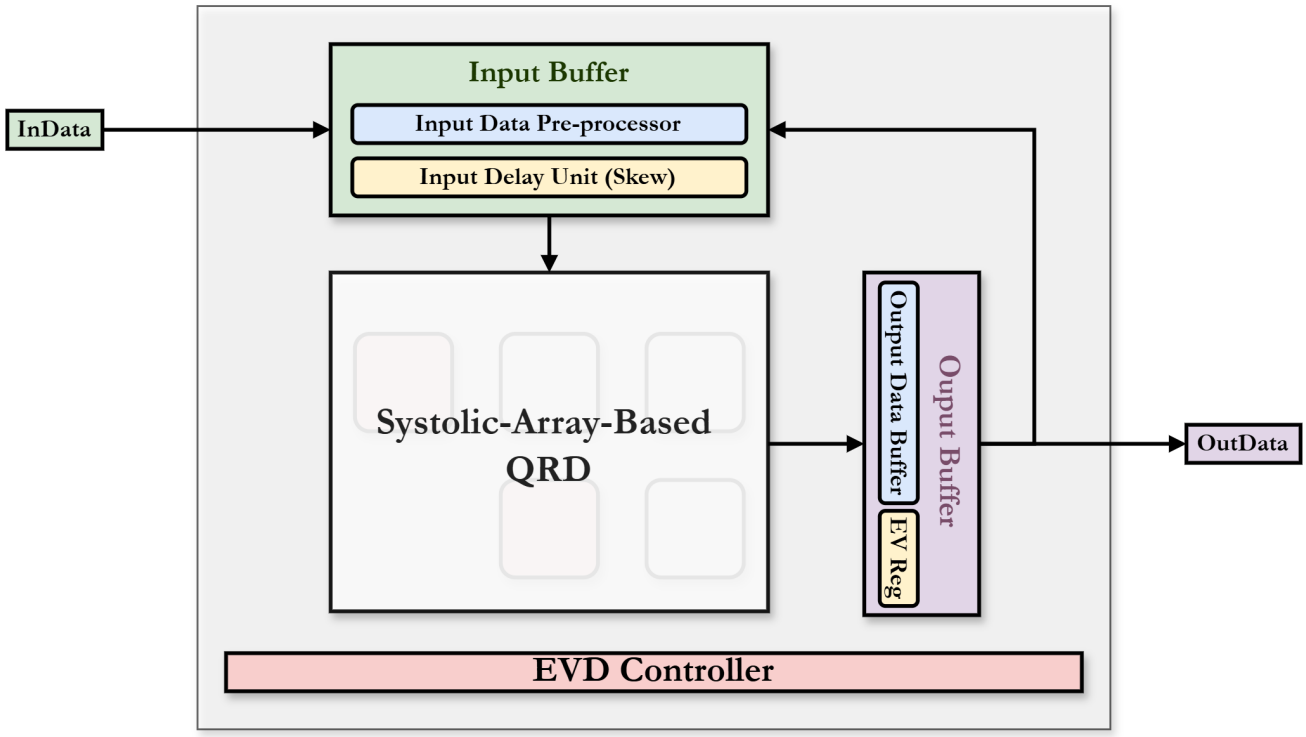

Figure 3.4: Top-level EVD architecture.

Section 4: Hardware Simulation

Section 5: Fixed-Point vs. Hardware Consistency

We compare RTL behavioral simulation results with the fixed-point bit-true model from Section 1.3. The outputs are consistent across all 11 patterns. Figure 5.1 shows pattern 8 as an example.

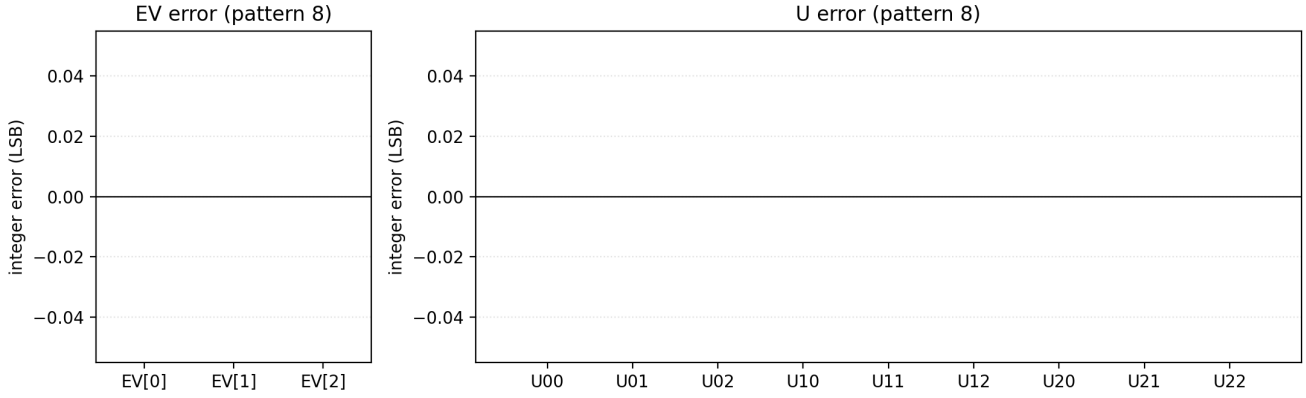

Figure 5.1: RTL vs. bit-true model error, pattern 8.

Section 6: Area-Time Product

Section 7: Design Innovations and Team Contributions

7.1 Team Contributions

Table 7.1: Team Contributions and Work Distribution

Work Item	**MING-HONG,LIN**	**YING-CHENG,CHEN**
Algorithm Research and Floating-Point Modeling	✓	✓
Bit-True Model Development	✓	✓
Hardware Word-Length and Accuracy Analysis	✓	✓
Hardware Architecture and Control Flow Design	✓	✓
RTL Design	✓	✓
Testbench Development and Unit Verification	✓	✓
Behavioral Simulation	✓	✓
Logic Synthesis and Area/Timing Analysis	✓	✓
Post-Synthesis Simulation	✓	✓
Hardware Optimization	✓	✓
Area-Time Product Evaluation	✓	✓
Result Analysis and Visualization	✓	✓
Report Writing	✓	✓